PROYECTO FINAL

32-MC3-011 Microcredencial Universitaria en Fundamentos Técnicos de la Ciberseguridad

Curso 2025-2026

Integración de un sistema de autenticación con Keycloak en una API REST desarrollada con FastAPI.

Óscar Gómez Fuente
2026-03-04

Contenido

1. Introducción 3
2. Instalación y despliegue de Keycloak con base de datos persistente 3
2.1. Crear Realm de trabajo 7
2.2. Crear los clientes necesarios 8
2.3. Crear los roles necesarios 11
2.4. Crear usuarios de aplicación 12
2.5. Esquema final configuración Keycloak 13
2.6. Verificar acceso Admin Console con el nuevo usuario 13
3. Probar los endpoints 14
3.1. Probar /health y /public 16
3.2. Probar /protected 17
3.3. Probar /admin {Realm role} 18
3.4. Probar /reports {Client role} 19
4. Verificar que el token está bien emitido 21
5. Obtener token utilizando POSTMAN 23
6. Copia de seguridad de la base de datos de Keycloak 24
7. Explicación FastAPI (main.py) 25
8. BONUS: Exportar configuración Keycloak 27

1. Introducción

Vamos a empezar realizado una introducción del trabajo seleccionado: **"Integración de un sistema de autenticación con Keycloak en una API REST desarrollada con FastAPI."**

Objetivo:

El objetivo de esta práctica es implementar un sistema completo de autenticación y autorización basado en estándar **OIDC** (OpenID Connect) utilizando **Keycloak** como **Identity Provider** (IdP) y **FastAPI** como backend.

La solución desarrollada permite:

- Autenticar usuarios mediante **Keycloak**.
- Validar tokens **JWT (JSON Web Token)** firmados con RS256 usando **JWKS (JSON Web Key Set)**.
- Proteger endpoints de la API según roles (Realm roles y Client roles).
- Integrar Swagger UI con Authorization Code Flow + PKCE, permitiendo login directamente desde /docs.

NOTAS:

- **JWT:** Token de autenticación para validar la API.
- **JWKS:** Conjunto de claves púbicas usado por FastAPI para validar firma JWT.

Todo el código generado se encuentra en el siguiente repositorio público de GitHUB:
https://github.com/oscargomezf/fastapi-keycloak-jwt-authentication

A continuación, se describen todos los pasos realizados:

2. Instalación y despliegue de Keycloak con base de datos persistente

Para disponer de una instalación realista y persistente, desplegamos ***Keycloak*** utilizando ***Docker Compose*** junto con una base de datos ***PostgreSQL***, lo cual permite conservar configuraciones, usuarios y roles al reiniciar el entorno.

Creamos un archivo ***docker-compose-keycloak.yml*** con los siguientes servicios:

- postgres (imagen postgres:15).
- keycloak (imagen quay.io/keycloak/keycloak:26.5)
- app-fastapi: main.py implementa una API ***FastAPI*** protegida con ***Keycloak*** que valida tokens ***JWT*** (RS256) vía ***JWKS*** y controla acceso por roles. También integra ***Swagger*** con ***OAuth2 Authorization Code*** para autenticación desde el navegador.

```
services:
  postgres:
    image: postgres:15
    container_name: postgres-service
    restart: unless-stopped
    env_file:
      - .env
    environment:
      TZ: Europe/Madrid
    ports:
      - "5432:5432"
```

```
    volumes:
      # Persistent PostgreSQL data directory
      - keycloak_pgdat:/var/lib/postgresql/data
      # Initialization scripts & dump (executed ONLY when the data directory is empty)
      - ../postgres-init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 60s
    networks: [appnet]   # [OPTIONAL-CHANGE]: connect to a common network (if you want
to use host.docker.internal on Windows/Mac)

  keycloak:
    image: quay.io/keycloak/keycloak:26.5
    container_name: keycloak-service
    restart: unless-stopped
    env_file:
      - .env
    #command: ["start-dev", "--hostname=host.docker.internal", "--hostname-strict=false",
"--health-enabled=true"]
    #command:  ["start-dev",  "--hostname=localhost",  "--hostname-strict=false",  "--
health-enabled=true"]
    command: ["start-dev", "--hostname-strict=false", "--health-enabled=true"]
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://postgres:5432/${POSTGRES_DB}
      KC_DB_USERNAME: ${POSTGRES_USER}
      KC_DB_PASSWORD: ${POSTGRES_PASSWORD}
      # Optional: give more margin to the pool while Postgres starts
      QUARKUS_DATASOURCE_JDBC_ACQUISITION_TIMEOUT: 60S
      # Optional: bootstrap admin to log in directly
      #KC_BOOTSTRAP_ADMIN_USERNAME: ${KC_BOOTSTRAP_ADMIN_USERNAME}
      #KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_BOOTSTRAP_ADMIN_PASSWORD}
      KEYCLOAK_ADMIN: ${KEYCLOAK_ADMIN}
      KEYCLOAK_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD}
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
    # Optional: Keycloak healthcheck (when already connected to the DB)
    healthcheck:
      # TCP check: Open connection to 127.0.0.1:8080; if it fails, exit 1
      test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8080 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 30
      start_period: 60s
    # Connect to a common network
    networks: [appnet]
```

```
  app-fastapi:
    build:
      context: ../app
      dockerfile: Dockerfile
    image: app-fastapi:latest
    container_name: app-fastapi-service
    restart: unless-stopped
    env_file:
      - .env
    depends_on:
      keycloak:
        condition: service_healthy
    ports:
      - "8001:8001"
    healthcheck:
      # TCP check: Open connection to 127.0.0.1:8001; if it fails, exit 1
      test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8001 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 30
      start_period: 60s
    # Connect to a common network
    networks: [appnet]

volumes:
  keycloak_pgdat:
    name: keycloak_pgdata

networks:
  appnet: {}
```

Revisar en GitHUB:
https://github.com/oscargomezf/fastapi-keycloak-jwt-authentication/blob/main/compose/docker-compose-keycloak.yml

Levantamos los servicios:

```
$ docker compose -f compose/docker-compose-keycloak.yml up
```

Apagamos los servicios:

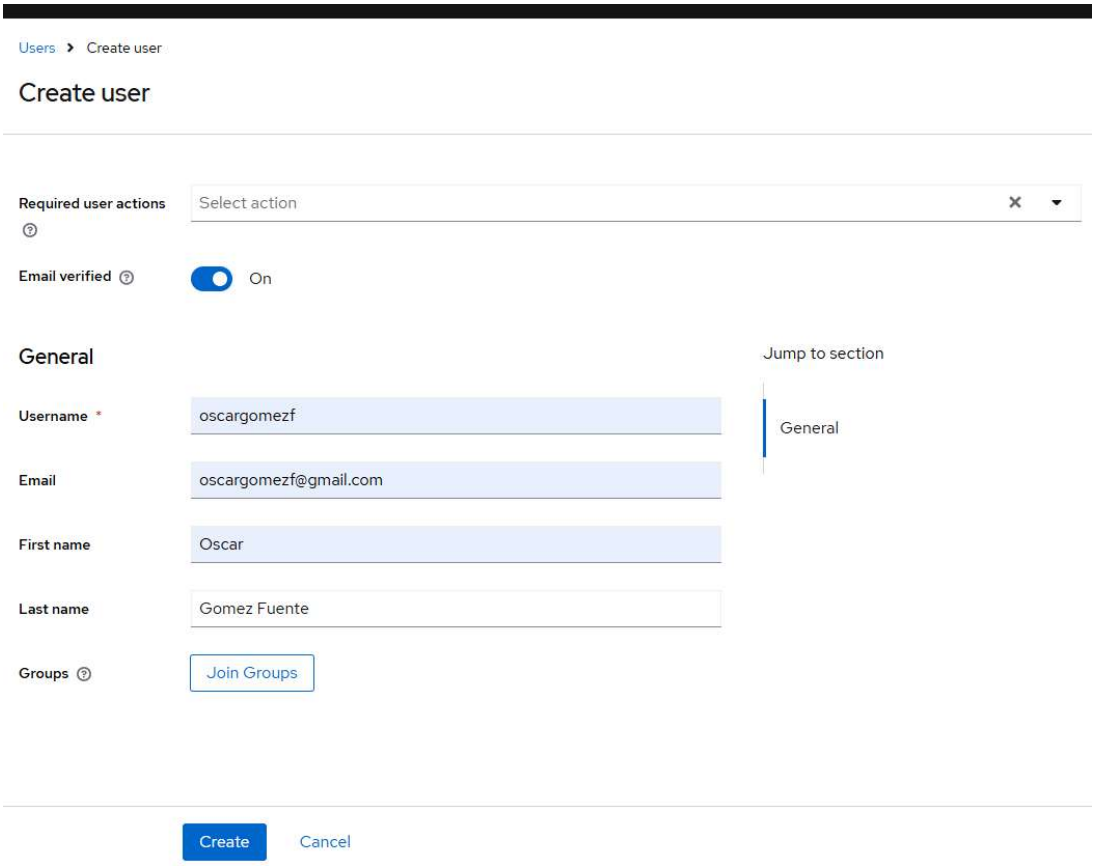
```
$ docker compose -f compose/docker-compose-keycloak.yml down -v
```

La primera vez de todas, tendremos que configurar keycloak, ya que viene con la configuración por defecto, pero una vez finalziado exportaremos la BBDD de postgres, para no tener que volver hacer el proceso de nuevo. Pasos a seguir:

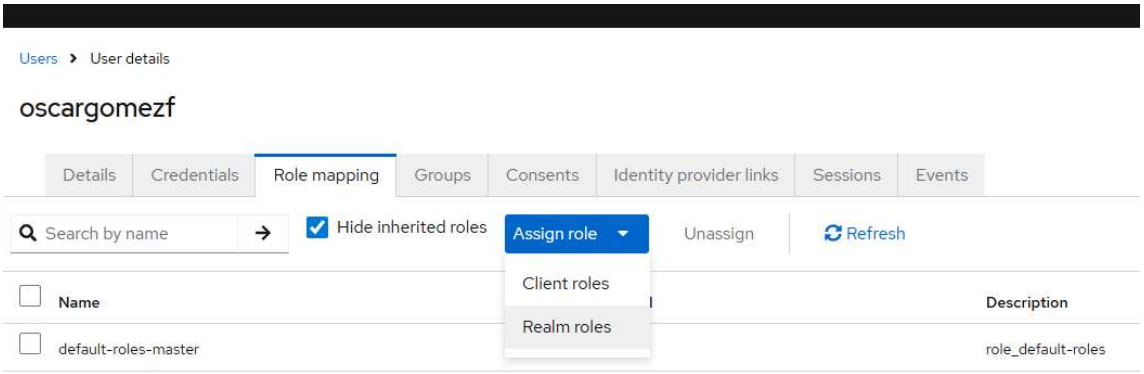
Accedemos a la consola de administración con ***http://localhsot:8080***

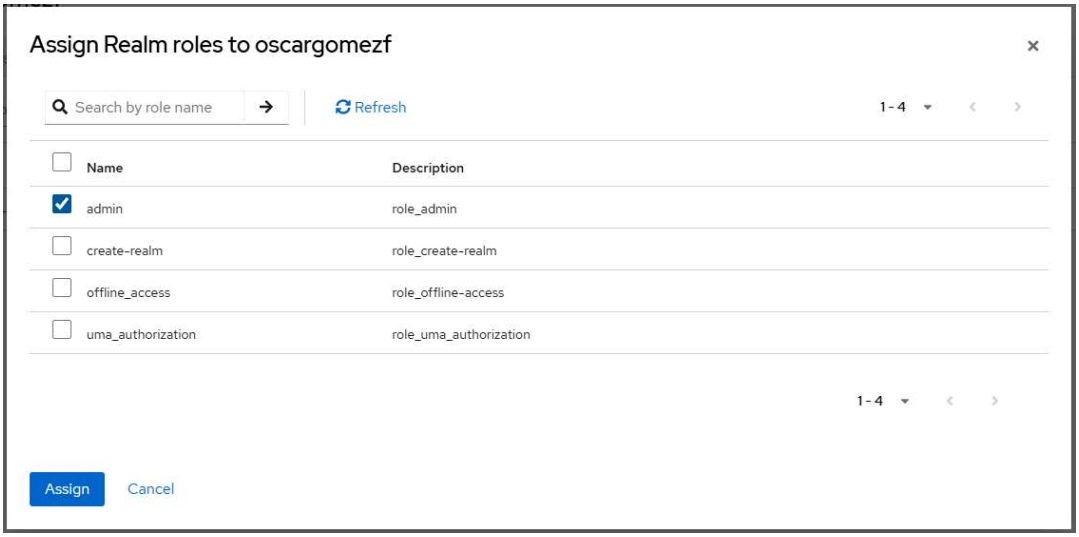
- User: admin
- Password: admin

Lo primero que tenemos que hacer es crear un usuario con permiso de administrados, con lo que creamos un nuevo usuario llamado: **oscargomezf**, y le añadimos permisos de administrador.



Como hemos comentado, añadimos al usuario ***oscargomezf*** el Realm role de administrador:

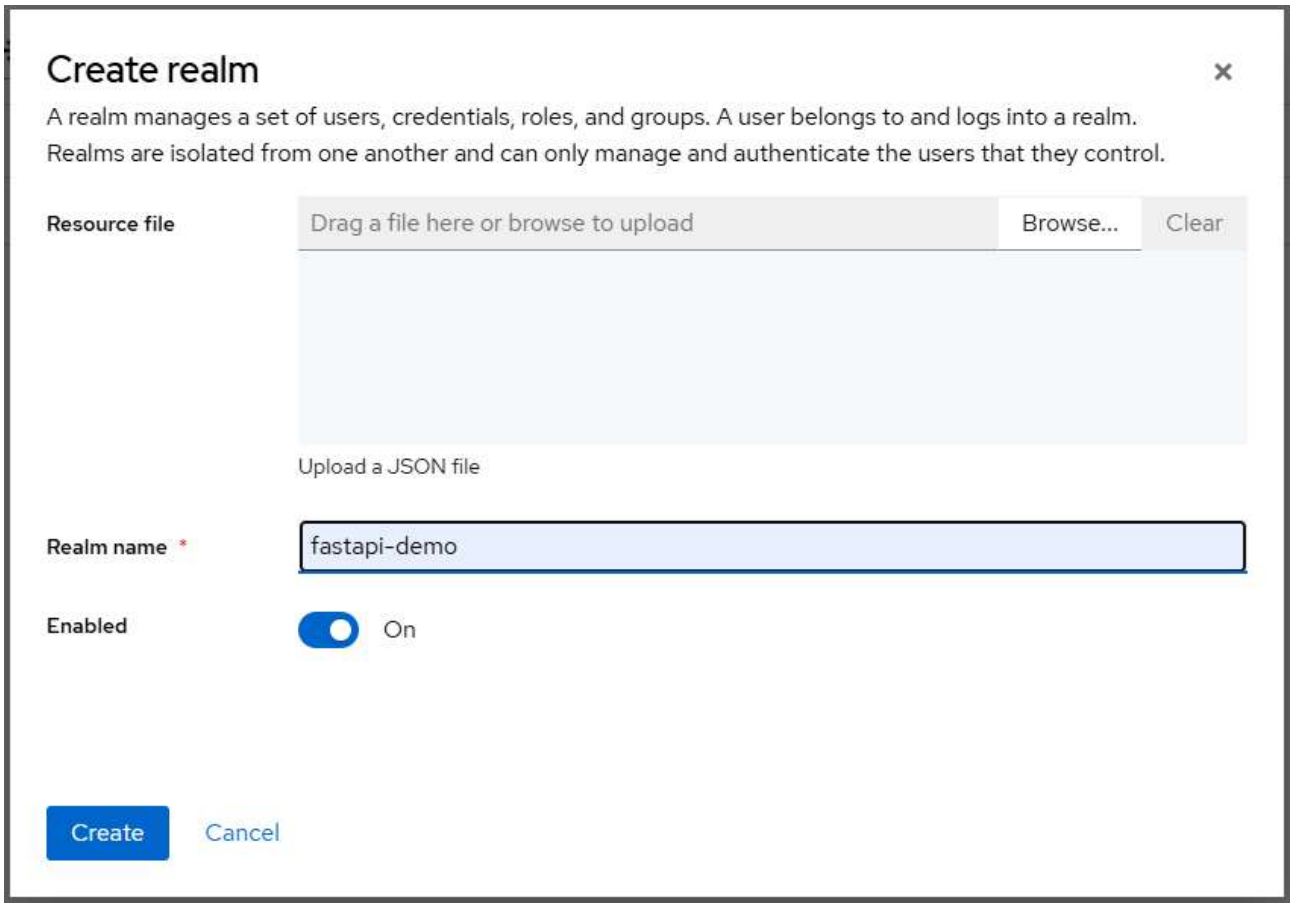




Por último, creamos un password para este usuario, y, a partir de aquí, utilizaré este usuario como administrador.

2.1. Crear Realm de trabajo

Como siguiente paso, procedemos a crear un **Realm de trabajo**, al que llamaremos ***fastapi-demo***:



Ahora, procedemos a crear un administrador del **Realm**, al que podemos llamar ***real-admin***.

Pasos a seguir:

1. En el realm ***fastapi-demo***, abrimos ***Users*** → ***i***.
2. Creamos un usuario, por ejemplo, username: ***realm-admin***.
3. En la sección ***Credentials***, añadimos la contraseña y desmarcamos ***Temporary***.

4. Como siguiente paso, accedemos a la sección: ***Role Mappings***.
5. Pulsamos ***Assign Role*** y filtramos por ***clients***.
6. Seleccionamos el ***realm-management***.
7. Y por último, asignamos el rol ***realm-admin.***

2.2. Crear los clientes necesarios

Una vez dentro de este nuevo realm, que hemos llamado ***fastapi-demo***, el siguiente paso, es crear los dos clientes que voy a necesitar:

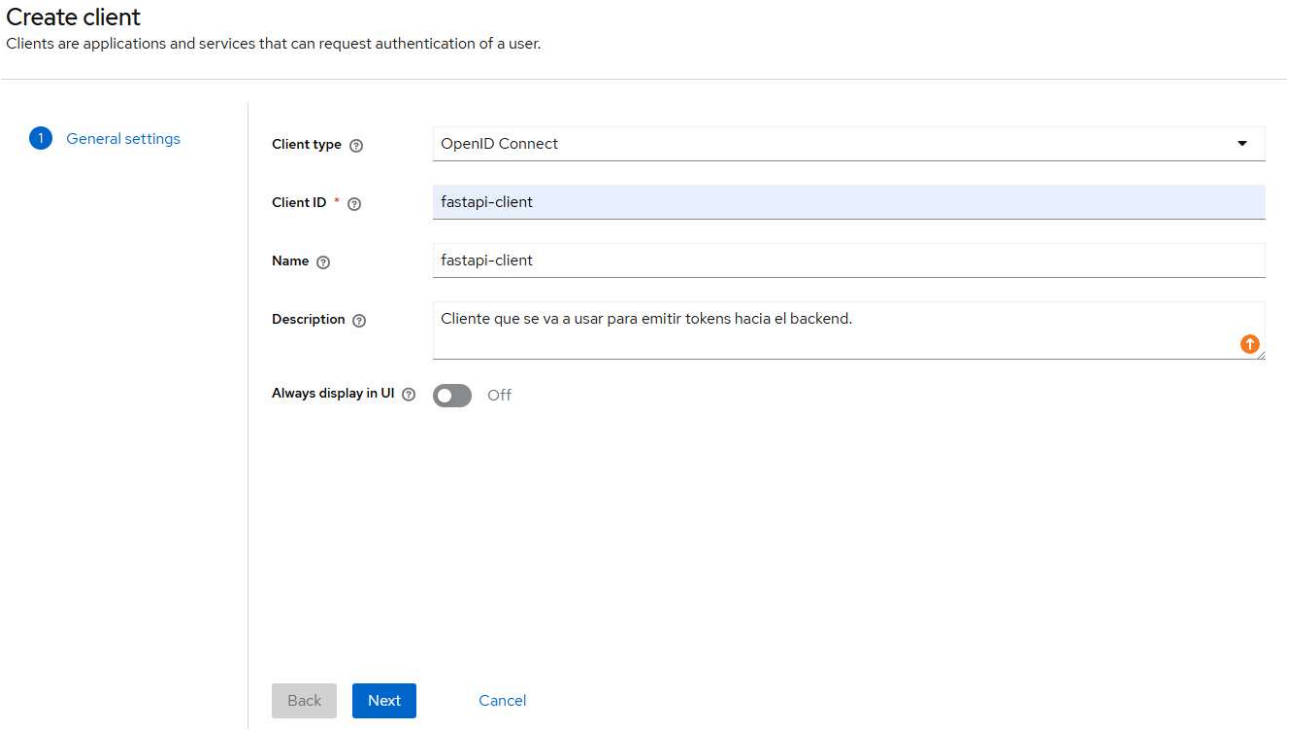
Cliente A: **fastapi-client** (CONFIDENTIAL). Este es el cliente usado por la API y por cualquier backend para emitir tokens mediante:

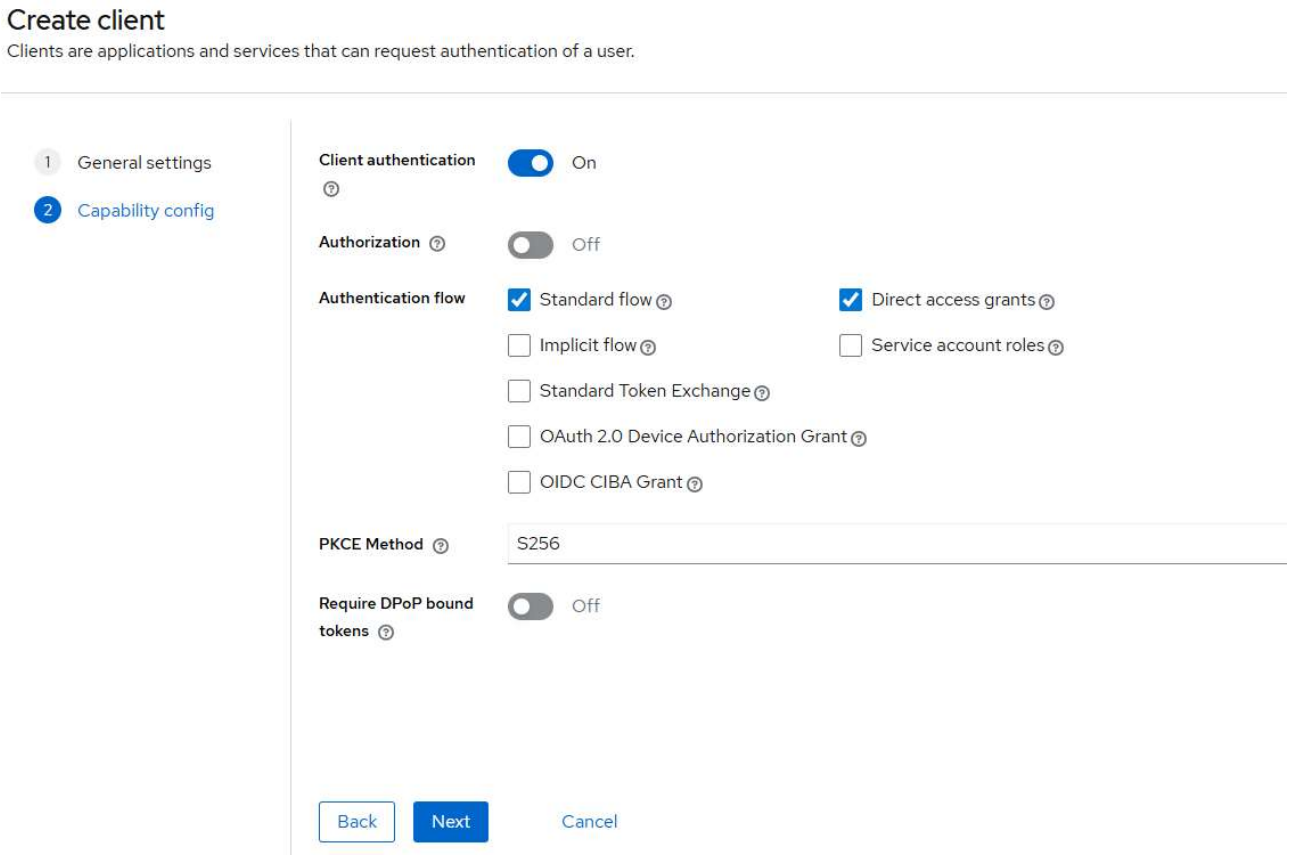
- Authorization code Flow.
- Direct Access Grants (solo en pruebas).

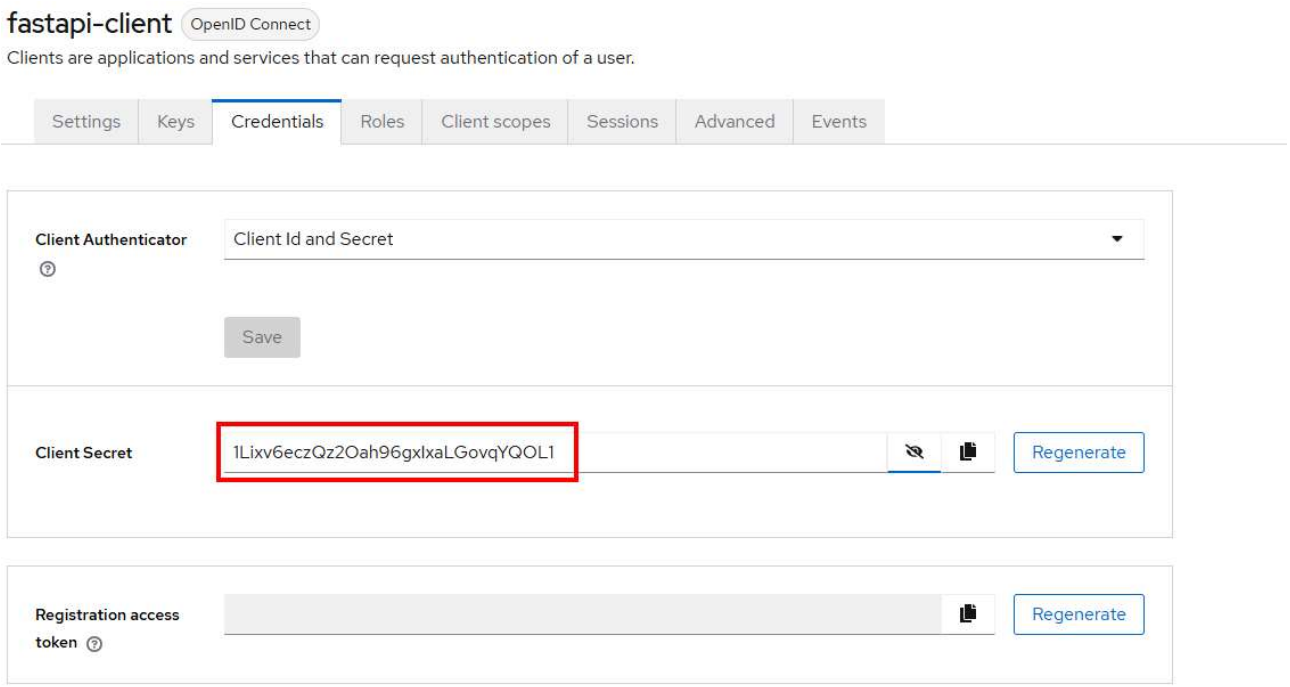
El cliente **fastapi-client**, se creará con la siguiente configuración:

- Client authentication: ON
- Standard Flow: ON
- Direct Access Grants: ON (solo para pruebas)
- PKCE Method: S256

NOTA: Guardamos el Client Secret







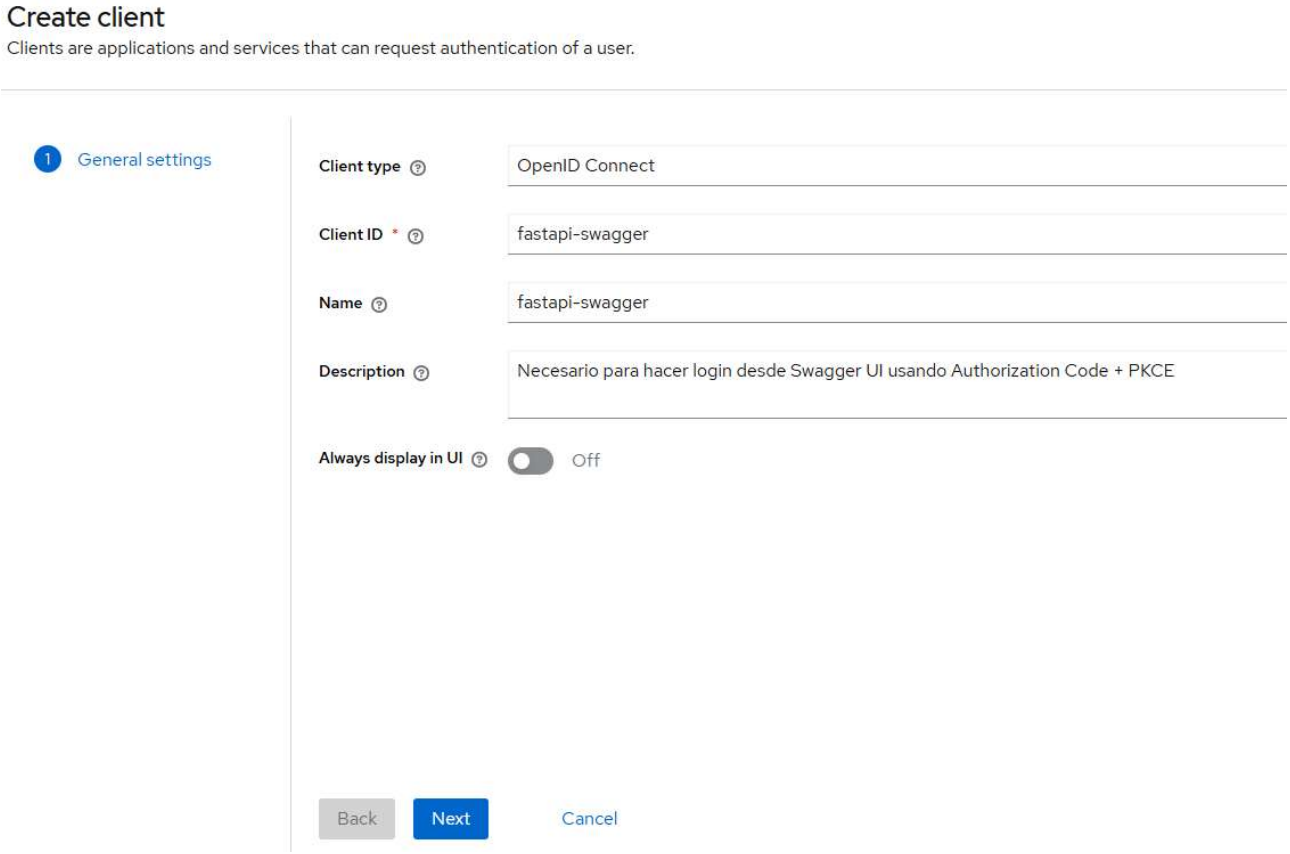
Client Secret: 1Lixv6eczQz2Oah96gxIxaLGovqYQOL1

Cliente B: **fastapi-swagger** (PUBLIC). Este cliente sirve solo para Swagger UI, para permitir que ***/docs*** pueda iniciar sesión en Keycloak mediante Authorization Code + PKCE.

Creamos el cliente **fastapi-swagger**, con la siguiente configuración

- Client authentication: OFF

- Standard Flow: ON
- PKCE Method: Vacío
- Redirect URI: http://localhost:8001/docs/oauth2-redirect
- Web Origins: http://localhost:8001
- Este usuario no tiene secret, ya que es público.



Create client

Clients are applications and services that can request authentication of a user.

1 General settings
2 Capability config
3 Login settings

Root URL

Home URL

Valid redirect URIs http://localhost:8001/docs/oauth2-redirect

Add valid redirect URIs

Valid post logout redirect URIs

Add valid post logout redirect URIs

Web origins http://localhost:8001

Add web origins

Back Save Cancel

2.3. Crear los roles necesarios

Ahora, procedemos a crear el Realm role que usará el endpoint ***/admin*** en ***FastAPI***. Para ello, hacemos click a ***Create Role*** en la sección ***Realm roles***:

Create role

Role name * admin

Description Creamos el Realm role que usará el endpoint /admin e FastAPI

Save Cancel

Ahora, creamos el role, **report:read**, del cliente **fastapi-client**, para ello, vamos a la sección ***Clients***, seleccionamos ***fastapi-client*** →***Roles***, y pulsamo ***en Create Role***. Este rol lo usará el endpoint ***/reports***.

Create role

Role name * report:read

Description Este rol lo usará tu endpoint /reports

Save Cancel

2.4. Crear usuarios de aplicación

Procedemos a crear un usuario de aplicación (no administrador). Este es el usuario que va a utilizar para autenticarse en FastAPI. Para ello, en el ***Menú Users .*** → ***Add user***

```
username: user_fastapi
password: user_fastapi
```

Create user

Required user actions Select action

Email verified Off

General

Jump to section

General

Username * user_fastapi

Email user_fastapi@gmail.com

First name User

Last name FastAPI

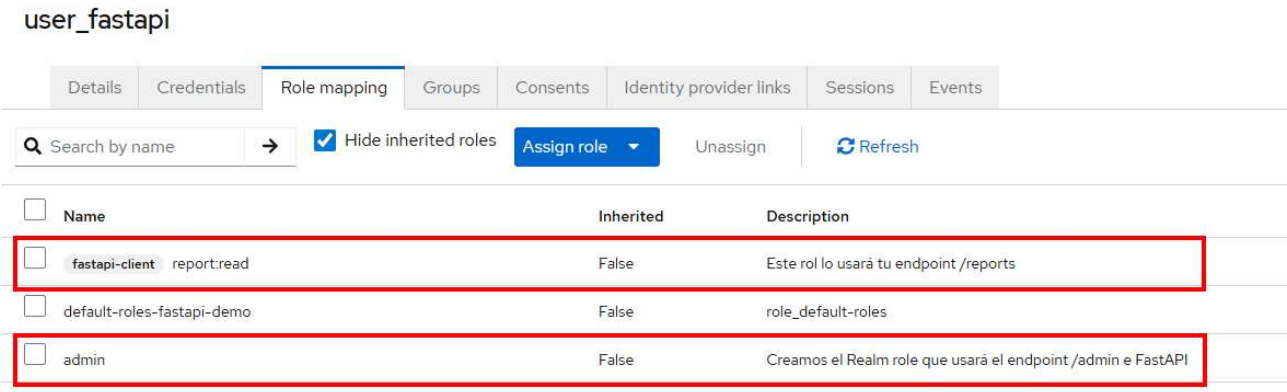
Groups Join Groups

Y en la pestaña ***Role mapping***, asignamos los siguientes roles a dicho usuario:

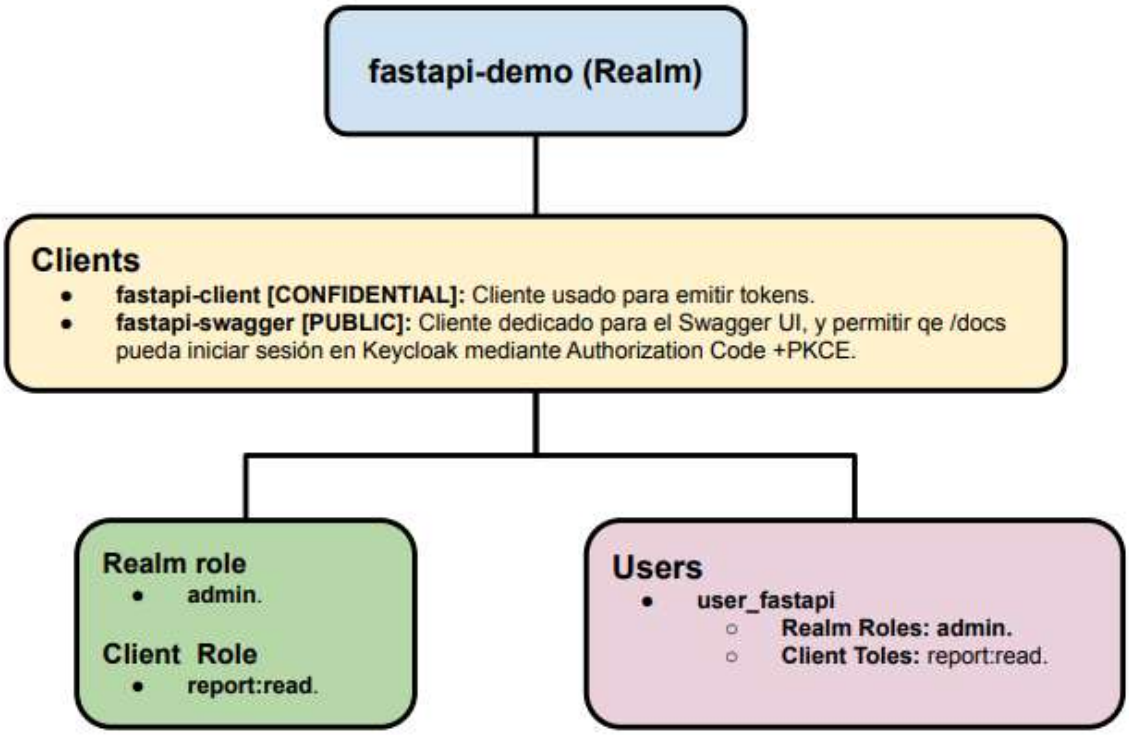
- Realm Roles: Asignar admin (si quieres que pueda entrar en /admin)
- Client Roles: Seleccionar fastapi-client y asignar report:read

Keycloak incluye los roles dentro del token JWT siguiendo su estructura estándar:

- Los **roles del realm** aparecen en realm_access.roles.
- Los **roles asignados a un cliente** se incluyen en resource_access.<client_id>.roles.



2.5. Esquema final configuración Keycloak



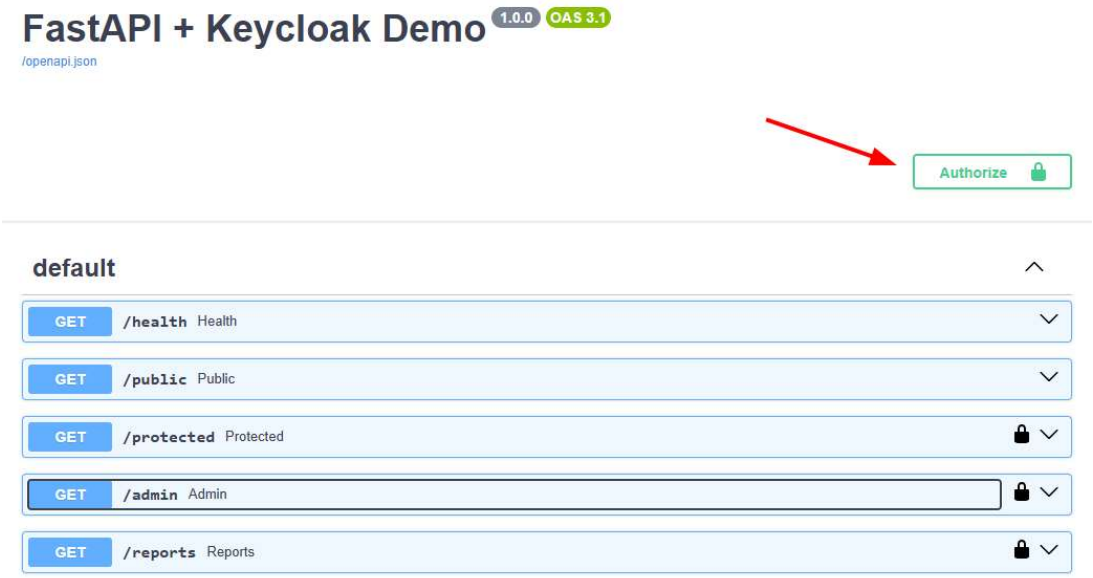
2.6. Verificar acceso Admin Console con el nuevo usuario

El usuario ***admin*** que hemos creado al inicio, ***oscargomezf*** debe poder acceder a la siguiente dirección: *http://localhost:8080/admin/fastapi-demo/console* ya que es administrador.

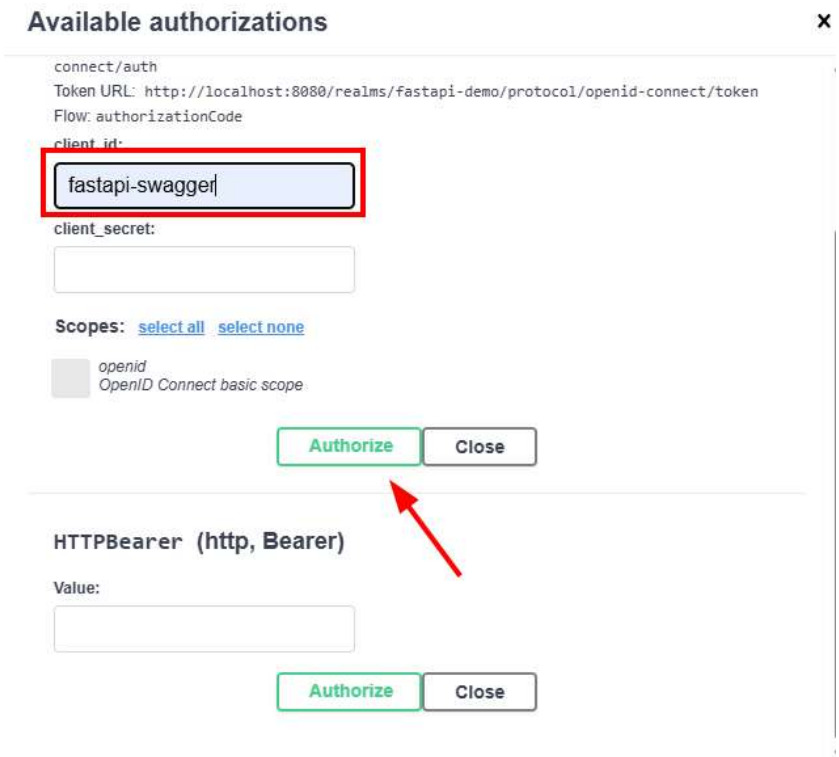
Esto es posible gracias al rol realm-admin, el cual se encuentra dentro del cliente interno realm-management.

3. Probar los endpoints

Antes de acceder a los recursos protegidos de la API, es necesario autenticarse. Para ello, en la interfaz Swagger UI pulsamos el botón "Authorize".

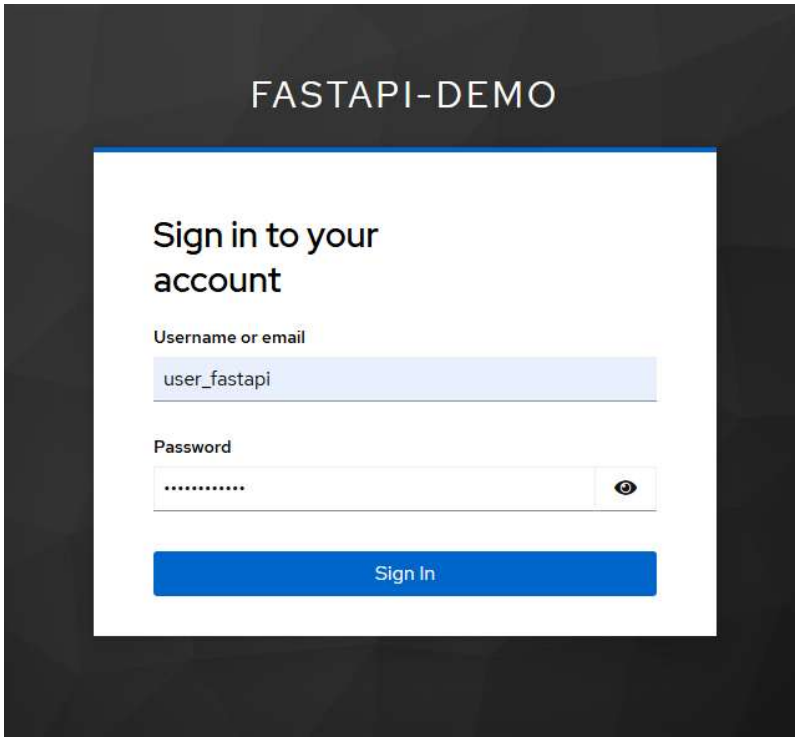


En este proceso utilizamos el cliente público configurado en Keycloak, denominado ***fastapi-swagger***, y hacemos clic en el botón **Authorize** del cuadro de diálogo.



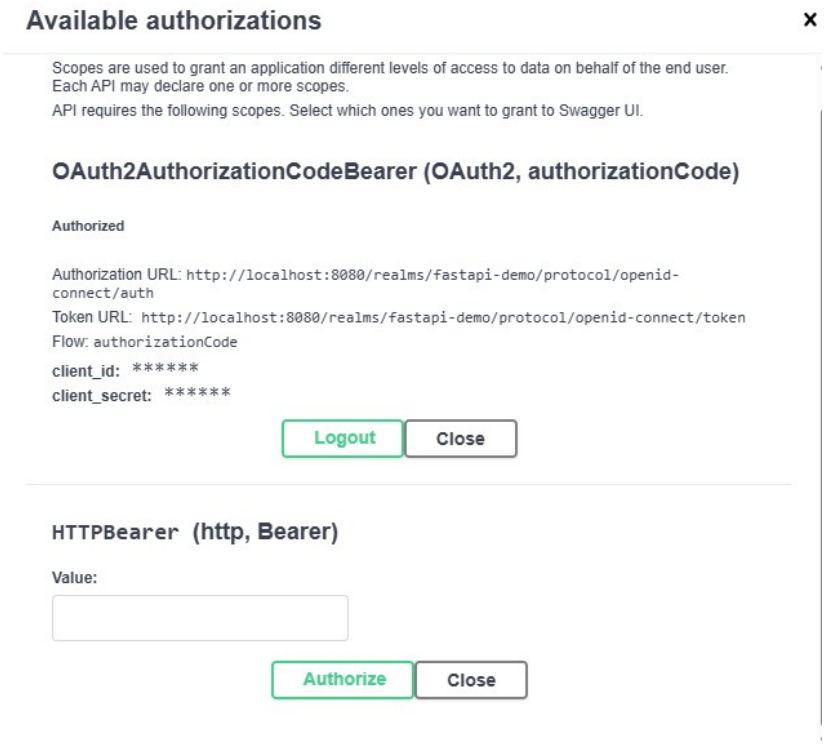
Al hacerlo, ***Swagger UI*** nos redirige automáticamente a la pantalla de autenticación de ***Keycloak***, donde introducimos las credenciales del usuario del realm, en este caso:

```
Username: user_fastapi
Password: user_fastapi
```



Una vez validadas las credenciales, ***Keycloak*** completa el flujo ***Authorization Code*** y devuelve el control nuevamente a ***Swagger UI*** en la ruta:

```
http://localhost:8001/docs/oauth2-redirect
```



Si todo ha ido correctamente, ***Swagger*** habrá almacenado el ***access token*** y podremos acceder a los recursos protegidos de la API, como:

- /protected
- /admin
- /reports

Recordamos que algunos recursos no requieren autenticación, como:

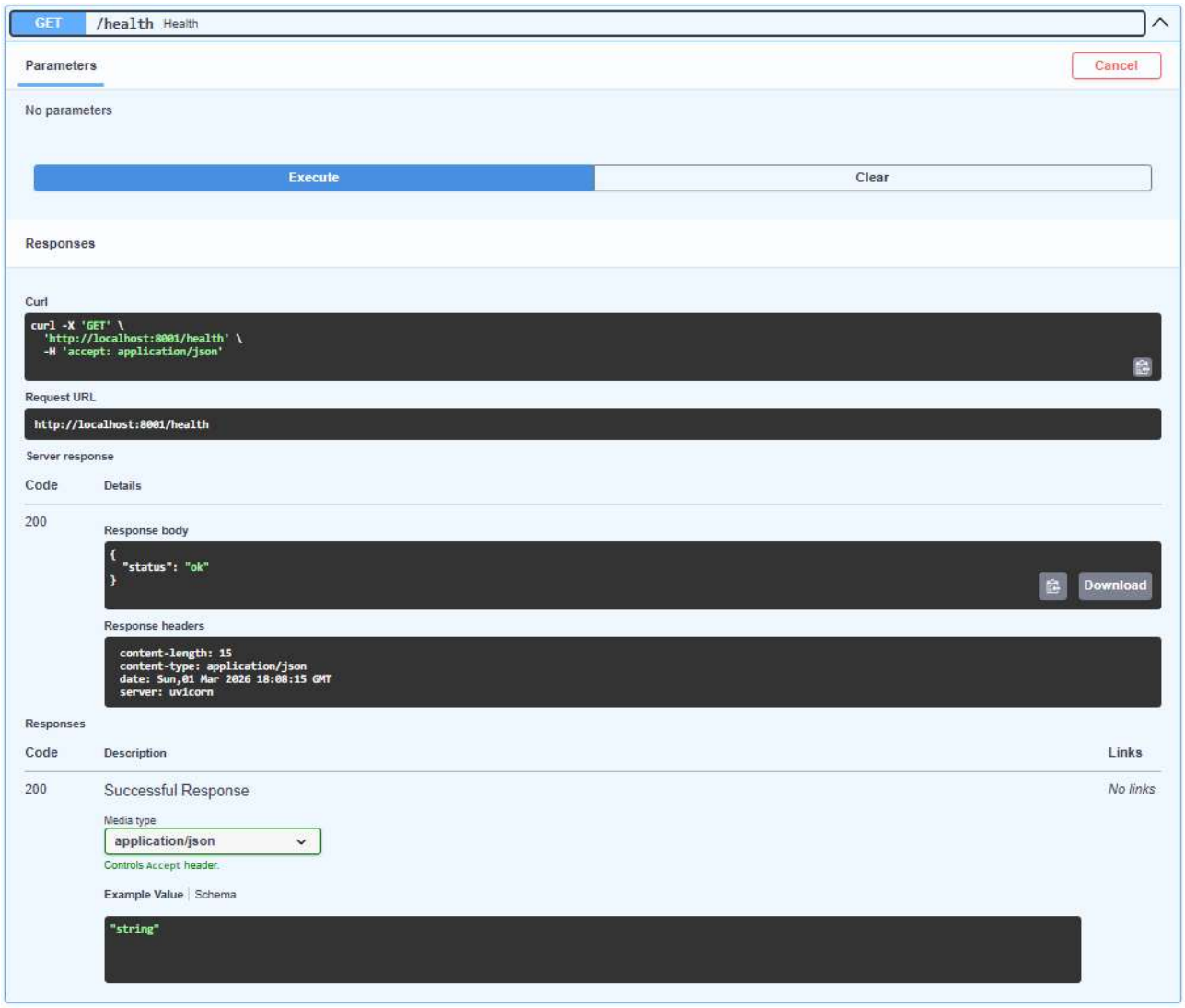
- /health
- /public

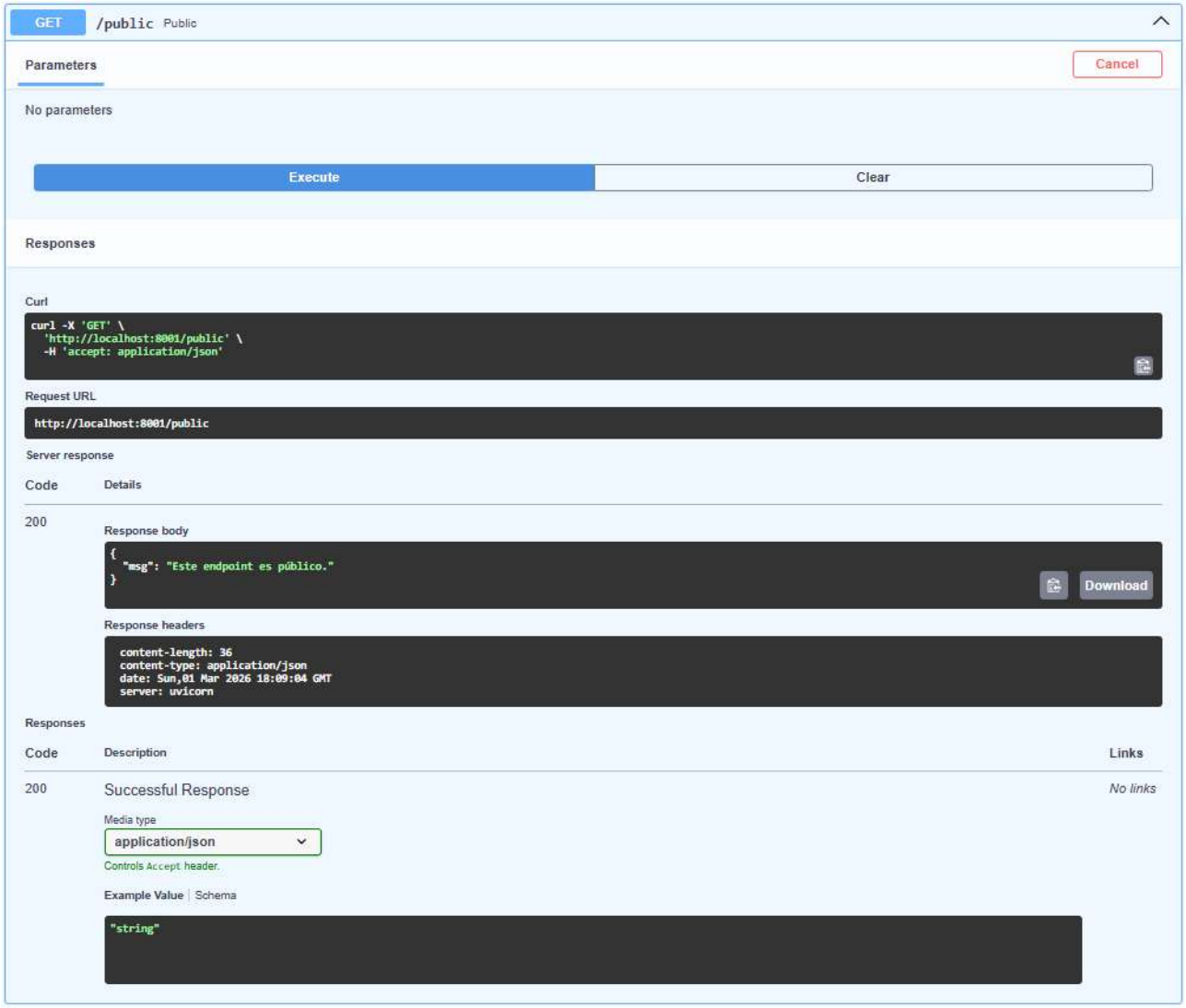
Estos son accesibles sin token.

3.1.Probar /health y /public

En Swagger, hay que hacer clic en en ***GET /public*** y pulsar ***Try it out → Execute***

Debe responder:





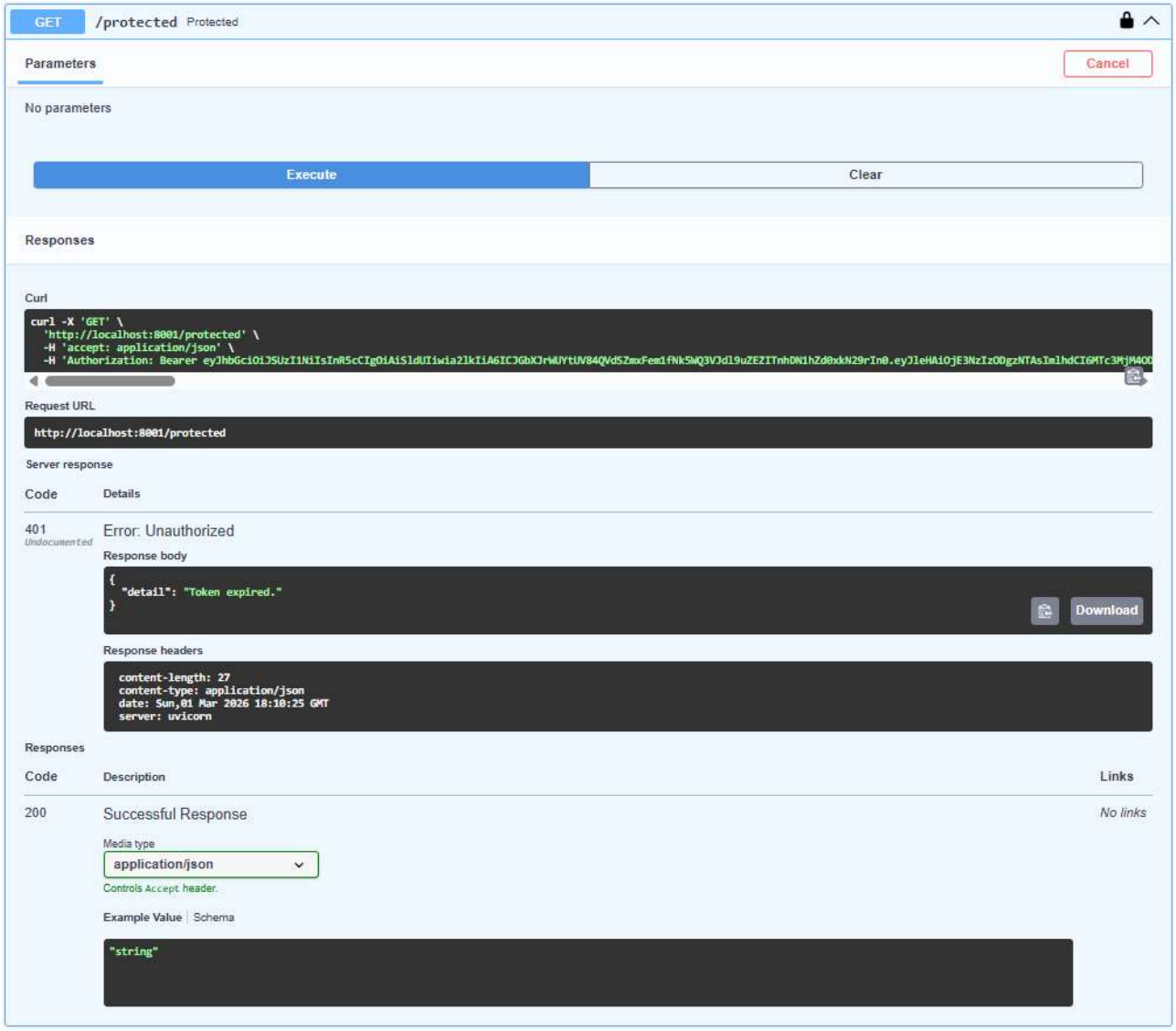
3.2.Probar /protected

Este endpoint verifica que:

- Swagger ha guardado el access token.
- FastAPI ejecuta verify_token correctamente.
- JWKS se descarga y valida la firma RS256 del token
- Los claims aud, iss, exp coinciden

Hay que pulsar:***GET /protected → Try it out → Execute***

Y se obtiene la siguiente salida:

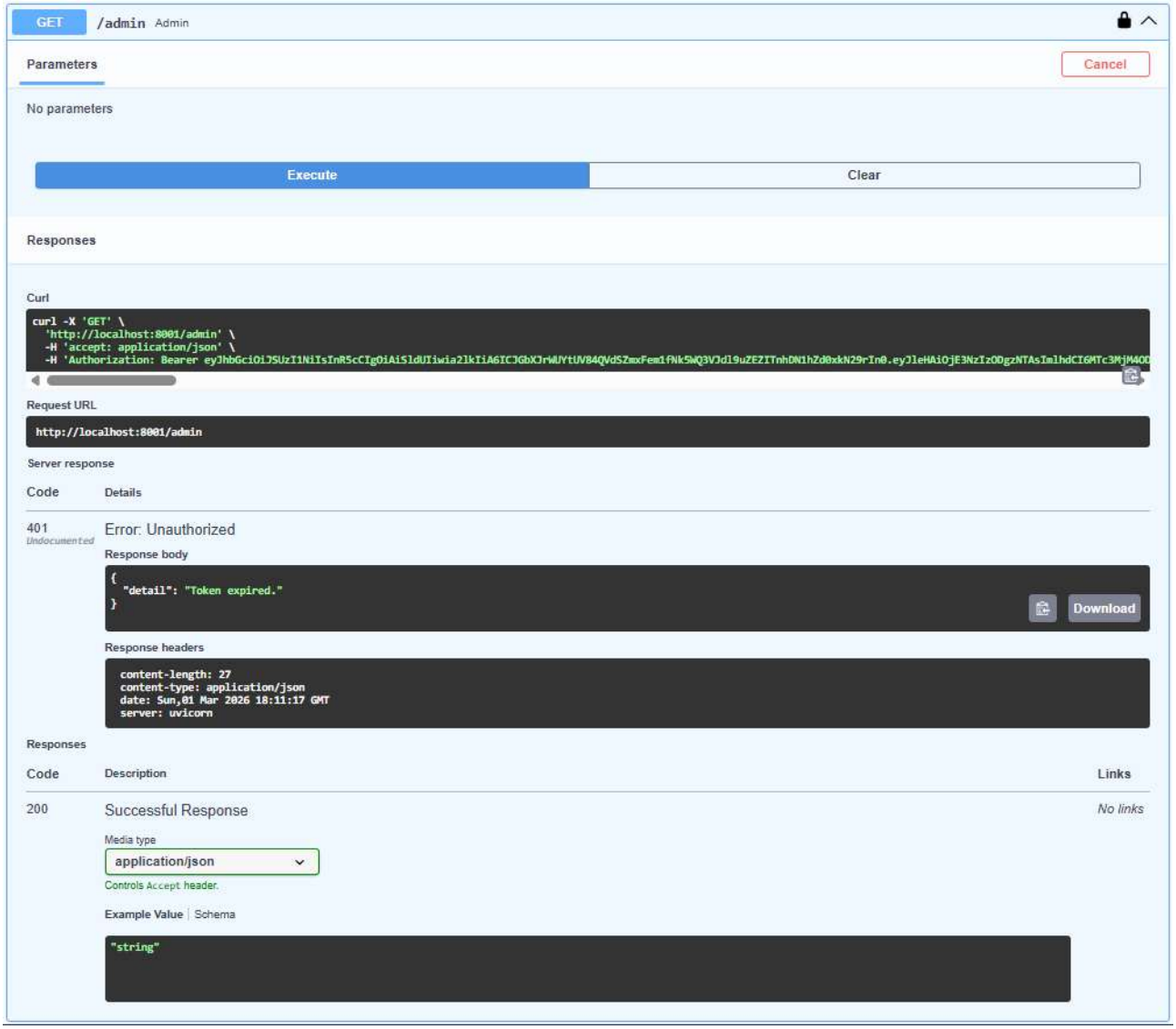


3.3.Probar /admin {Realm role}

Este endpoint exige el rol: ***realm_access.roles = ["admin"]***

Hay que hacer clic en ***GET /admin → Try it out → Execute***

Y el resultado esperado es el siguiente, si el usuario tiene el rol ***admin:***

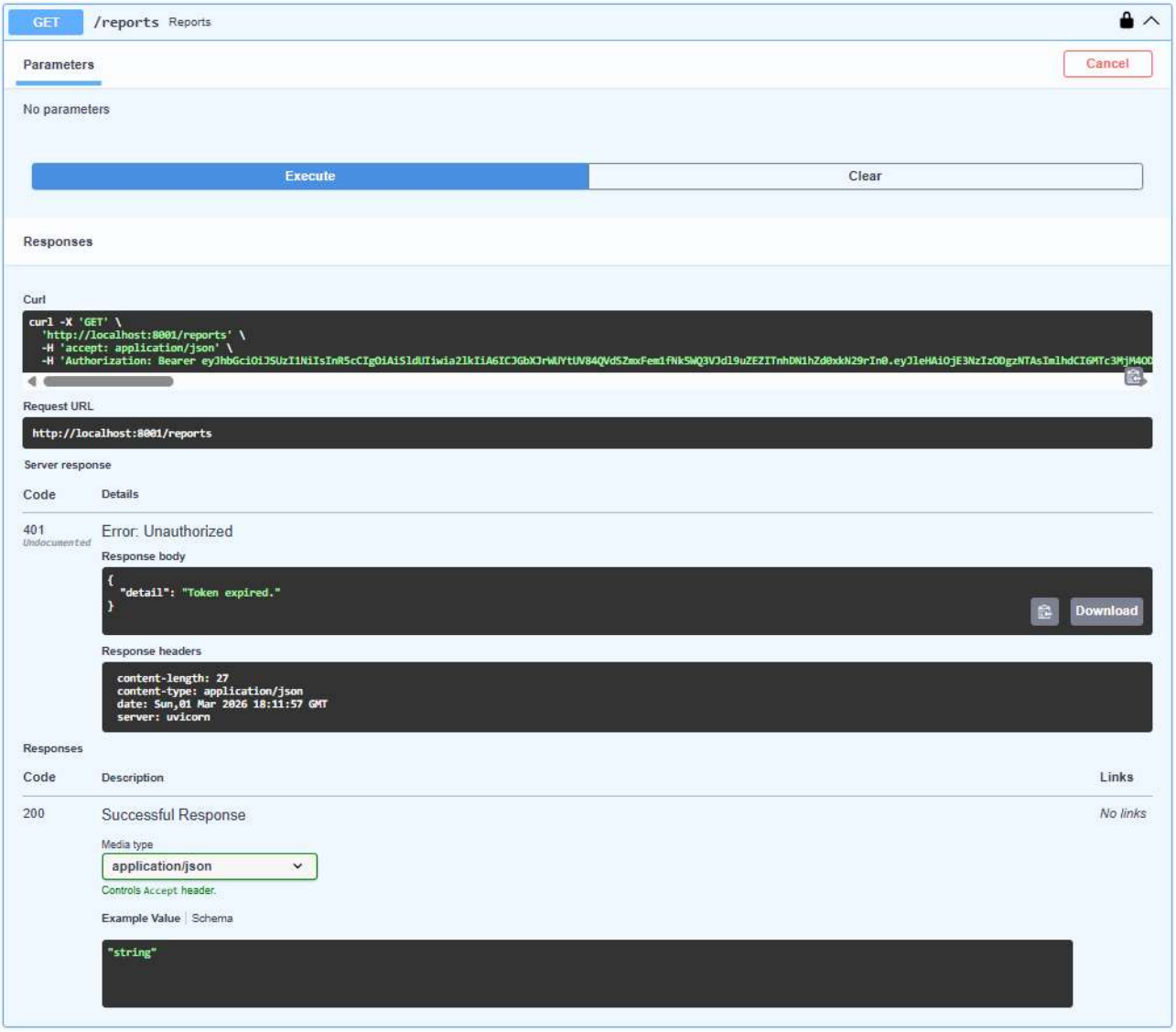


3.4.Probar /reports {Client role}

Este endpoint exige: ***resource_access.fastapi-client.roles = ["report:read"]***

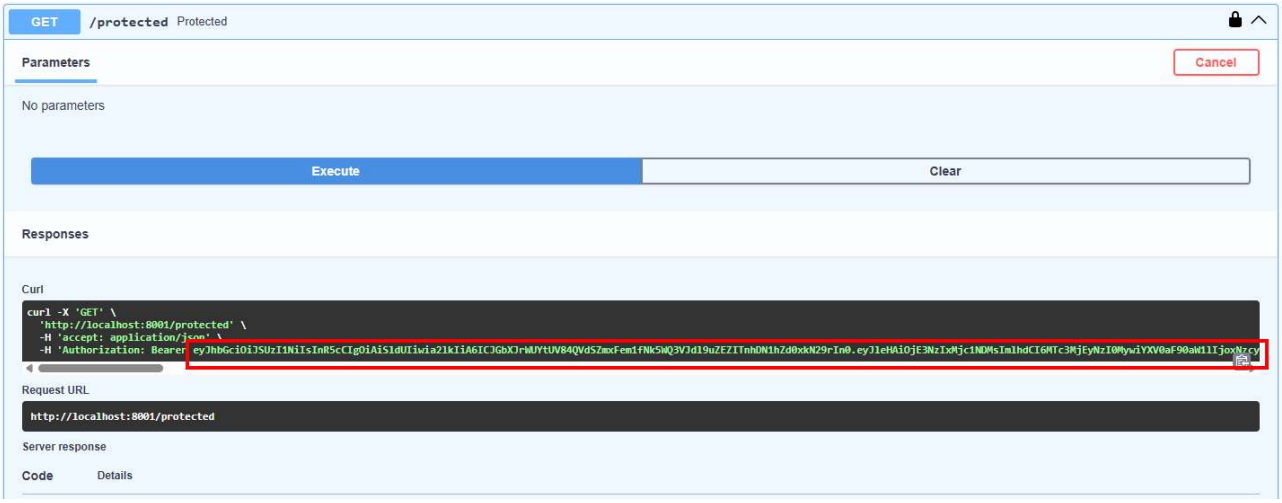
Hay que hacer click en ***GET /reports → Try it out → Execute***

Y responde lo siguiente:

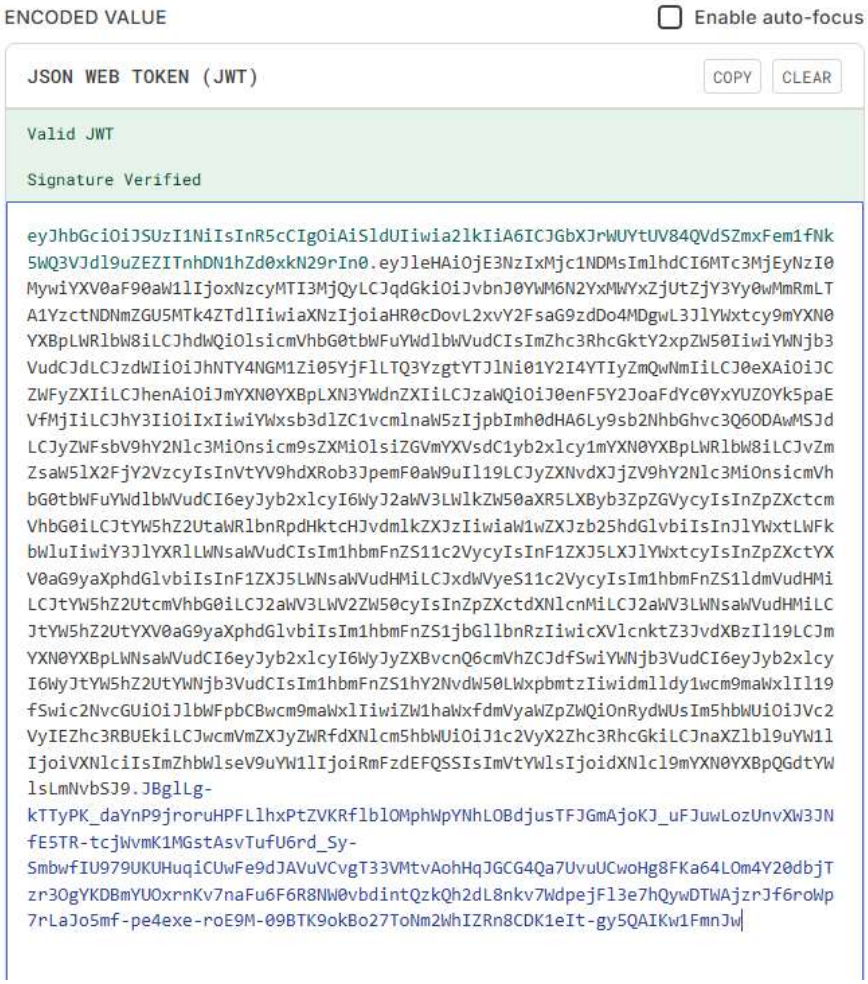


NOTA: La estructura ***resource_access.{client_id}.roles*** es la recomendada por Keycloak para roles de cliente.

4. Verificar que el token está bien emitido Por ejemplo, con el usuario ***user_fastapi***, podemos validar que el token se ha generado correctamente. Para ello, podemos copiar el token desde el endpoint /protected:



Y le decodificamos desde esta WEB: https://jwt.io



Comprobamos los siguientes campos:

- Decoded Header.
- Decoded Payload.
- Public Key.

DECODED HEADER

JSON CLAIMS TABLE COPY


```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "FmrkYF-Q_8AWRflEzm_6NVCuIv_ndFHNxC7XYwLd7ok"
}
```


DECODED PAYLOAD

JSON CLAIMS TABLE COPY


```
{
  "exp": 1772127543,
  "iat": 1772127243,
  "auth_time": 1772127242,
  "jti": "onrtac:7f11f1f5-f67c-02df-05c7-43fde9198e7e",
  "iss": "http://localhost:8080/realms/fastapi-demo",
  "aud": [
    "realm-management",
    "fastapi-client",
    "account"
  ],
  "sub": "a5684c5f-9b1e-47c8-a2e6-5cb8a22fd06b",
  "typ": "Bearer",
  "azp": "fastapi-swagger",
  "sid": "tzqycbhhWXsF1aFNbNihE_22",
  "acr": "1",
  "allowed-origins": [
    "http://localhost:8001"
  ],
  "realm_access": {
    "roles": [
      "default-roles-fastapi-demo",
      "offline_access",
      "uma_authorization"
    ]
  },
  "resource_access": {
    "realm-management": {
      "roles": [
        "view-identity-providers",
        "view-realm",
        "manage-identity-providers",
        "impersonation",
        "realm-admin",
        "create-client",
        "manage-users",
        "query-realms",
        "view-authorization",
        "query-clients",
        "query-users",
        "manage-events",
        "manage-realm",
        "view-events",
        "view-users",
        "view-clients",
        "manage-authorization",
        "manage-clients",
        "query-groups"
      ]
    },
    "fastapi-client": {
      "roles": [
        "report:read"
      ]
    },
  "scope": "email profile",
  "email_verified": true,
  "name": "User FastAPI",
  "preferred_username": "user_fastapi",
  "given_name": "User",
  "family_name": "FastAPI",
  "email": "user_fastapi@gmail.com"
}
```


PUBLIC KEY:


```
{
      "e": "AQAB",
      "kty": "RSA",
      "n": "tfxi_tUXfVEcOm398sAaxCEpu-RyQj-rhVQQqdjqRkLI0tXogZs9axd1eShscD2TCUiohUSaYhsOX8OijnT923BcoVK53Fek_Ru5vPYuEPE2p1C-3IxaOkd2arUHszGp8g2BqR2PDQYX2z7ni__dMur9O59Jrk8E27S4AJx01GBS-WDTwexVum2b7tVmTeo-TPb_xunvkf7FUvtIJmQma00RAPRCXgv8P5LE1kb-o9e4OavqlBazpAtq_csfM0KQB1_hMSAxpBT_QW0ynssXfkAk9c0W4rAoCLhAYXUpECyrKPZaT7dfs-a89V6mtPSHnOpbtfpGmR7ozllW8-C19w"
}
```

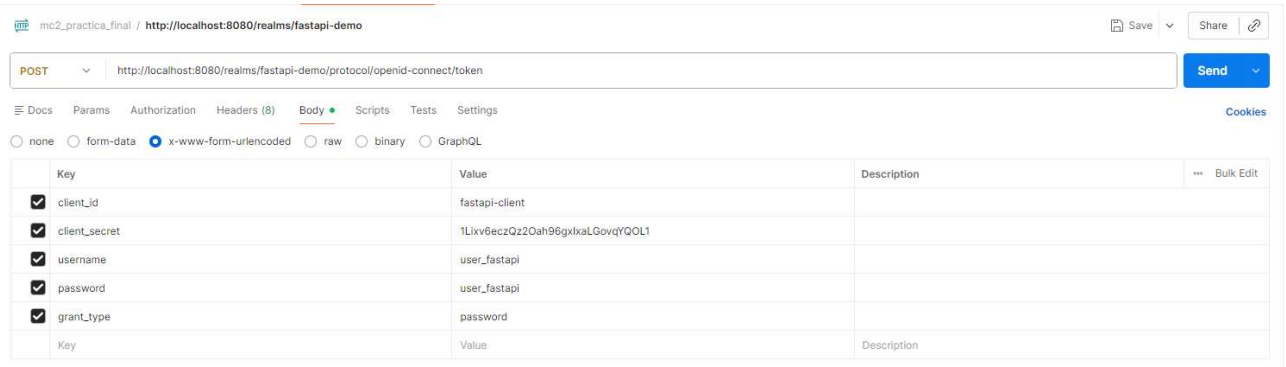
5. Obtener token utilizando POSTMAN

Otra forma de obtener el token, aunque es una forma más manual, es utilizar postman. Para ello, seguimos los siguientes pasos:

1. Hacer una petición para generar un token de acceso válido para el usuario ***user-fastapi*** del cliente fastapi-client:

```
http://localhost:8080/realms/fastapi-demo/protocol/openid-connect/token
```

2. Con estos parámetros en el body en modo fx-www-form-urlencoded:
   - client_id: fastapi-cliente
   - client_secret: 1Lixv6eczQz2Oah96gxIxaLGovqYQOL1
   - username: user_fastapi
   - password: user_fastapi
   - grant_type: password

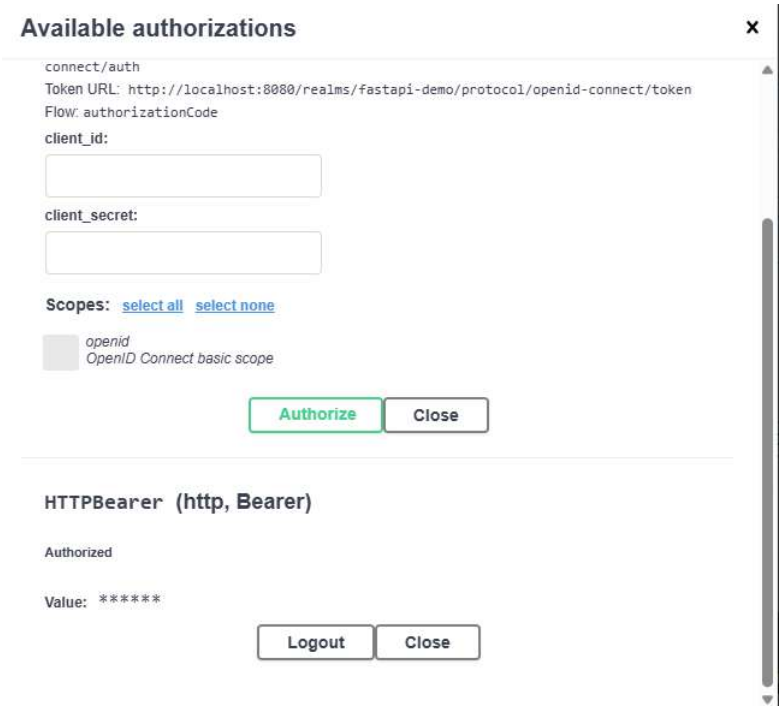


3. Y al enviar, vemos como obtenemos el access_token:

El access-token completo sin comillas, que consta de tres partes separadas por un punto:

HEADER.PAYLOAD.SIGNATURE

Es el token que tendremos que utilizar en el Swagger, en la sección HTTPBearer (http, Bearer), para conseguir acceso a lal API REST:



6. Copia de seguridad de la base de datos de Keycloak

Con el objetivo de no tener que repetir todos los pasos de generación de usuarios, clientes, real,...etc. Voy a proceder con crear una copia de seguridad de la BBDD keycloak, de forma, que, si se borrar la imagen, no sea necesario tener que volver a realizar todos estos pasos.

Para ello, procedo de la siguiente forma:

```
$ docker exec -i postgres-service \
pg_dump -U keycluoak -Fc keycloak > keycloak_backup.dump
```

Este comando crea el archivo ***keycloak_backup.dump*** en mi host. Y para restaurar el backup en un nuevo contenedor, procedo de la siguiente forma:

1. Copia el archivo al nuevo contenedor

```
$ docker cp keycloak_backup.dump postgres-service:/tmp/keycloak_backup.dump
```

2. Crear la base de datos vacía (No es necesario)

```
$ docker exec -it postgres-service \
createdb -U keycloak keycloak
```

3. Restaurar la BBDD

```
$ docker exec -it postgres-service \
pg_restore -U keycloak -d keycloak /tmp/keycloak_backup.dump
```

IMPORTANTE: Este dump de la BBDD de ***PostgreSQL*** lo hemos automatizado dentro del ***docker-compose***. La imagen oficial de ***PostgreSQL*** tiene una funcionalidad muy importante, si el directorio de datos (primer volumen) está vacío, PostgreSQL ejecuta automáticamente todos los scripts y dumps que encuentre en:

`/docker-entrypoint-initdb.d/`

Esto ocurre SOLO la primera vez que inicializamos el volumen. El script se ejecutará automáticamente, en este orden:

1. Se inicializa el clúster de Postgres vacío.
2. Se ejecuta restore.sh.
3. El dump keycloak_backup.dump se restaura.
4. Keycloak arranca con la base de datos ya configurada.

Ver los siguientes ficheros en el repositorio de GitHUB:

- ../compose/docker-compose-keycloak.yml
- ../compose/.env
- ../ postgres-init/keycloak_backup.dump
- ../ postgres-init/restore.sh
- ../ app/Dockerfile
- ../ app/main.py
- ../docs/c2_proyecto_final_sistema_autenticacion_keycloak_API_REST.docx.pdf

7. Explicación FastAPI (main.py)

En esta sección, voy a proceder a realizar una explicación detallada de la ***FastAPI*** implementada.

El código está disponible en este link de GitHUB:
https://github.com/oscargomezf/fastapi-keycloak-jwt-authentication/blob/main/app/main.py

Objetivo de la API

- Proveer una API REST protegida por Keycloak usando el estándar OIDC.
- Validar JWT RS256 localmente (por JWKS), sin introspección.
- Autorizar el acceso por roles (de realm y de cliente).
- Facilitar las pruebas desde Swagger UI (Authorization Code; PKCE opcional).

Seguridad (cómo protege)

1. Autenticación OIDC (Authorization Code):
   - El usuario hace login en Keycloak y obtiene un access_token (JWT).
   - Swagger UI gestiona el flujo y guarda el token para las llamadas a la API.

2. Validación de JWT en la API (local):
   - Extrae el kid de la cabecera del JWT.
   - Descarga las claves públicas (JWKS) de Keycloak y selecciona la que coincide con el kid.
   - Verifica firma (RS256), issuer (iss), audiencia (aud), expiración (exp), etc.
   - Si todo es correcto, el token se acepta y se leen los roles desde los claims.
3. Autorización por roles:
   - Realm roles en realm_access.roles (p. ej. admin).
   - Client roles en resource_access.<client_id>.roles (p. ej. report:read para fastapi-client).

Nota: Dejamos sin forzar la verificación de ***azp*** para permitir tokens emitidos a fastapi-swagger cuando la API espera fastapi-client como aud.

Arquitectura y variables clave

Clientes OIDC en Keycloak:

- fastapi-client (confidential): emisión de tokens (scripts/servicios).
- fastapi-swagger (public): login desde Swagger UI (Authorization Code; PKCE opcional).

Separación de URL pública e interna (útil si hay Docker/Reverse Proxy):

- KEYCLOAK_PUBLIC_URL: Lo que ven navegador/Swagger.
- KEYCLOAK_INTERNAL_URL: Lo que usa la API para JWKS.

Issuer & JWKS:

- PUBLIC_ISSUER = <KEYCLOAK_PUBLIC_URL>/realms/<realm> (se valida iss contra este).
- JWKS_URL = <KEYCLOAK_INTERNAL_URL>/realms/<realm>/protocol/openid-connect/certs (para firma).

Otras env:

KEYCLOAK_REALM, KEYCLOAK_AUDIENCE (p. ej. fastapi-client), CLIENT_ID_FOR_ROLES, ACCEPTED_ALGS=['RS256'].

Endpoints que expone

Abiertos

- GET /health → liveness
- GET /public → acceso sin token

Protegidos (requieren token válido)

- GET /protected → usuario autenticado (muestra preferred_username, email)
- GET /admin → requiere realm role admin
- GET /reports → requiere client role report:read en fastapi-client

Debug opcionales (para diagnóstico)

- GET /debug/config → muestra configuración efectiva (public/internal urls, jwks, issuer…)
- GET /debug/status → prueba de acceso a JWKS

Flujo de uso (Swagger y Postman)

Swagger UI

1. Abres http://localhost:8001/docs → Authorize.
2. Login en Keycloak (realm fastapi-demo) con el usuario de pruebas.
3. Swagger guarda el token y puedes llamar a /protected, /admin, /reports.
    - Con PKCE (opcional): activar en Keycloak (Public client) y en Swagger (initOAuth).

Postman / cURL

- POST a /protocol/openid-connect/token con grant_type=password, client_id=fastapi-client, client_secret, username, password.
- Pegar solo el access_token en Authorization: Bearer <token> para consumir la API.

Reglas de autorización (qué mira la API)

- /protected: token válido (firma, iss=PUBLIC_ISSUER, aud contiene fastapi-client, no expirado).
- /admin: además, admin ∈ realm_access.roles.
- /reports: además, report:read ∈ resource_access["fastapi-client"].roles.

Mini-diagrama (texto) del flujo

```
[Usuario] --(Authorize)--> [Swagger UI]
[Swagger UI] --(OIDC /auth)--> [Keycloak]
[Keycloak] --(code)--> [Swagger UI] --(token)--> [Keycloak /token]
[Swagger UI] --(Authorization: Bearer <JWT>)--> [FastAPI]
[FastAPI] --(JWKS)--> [Keycloak certs]
[FastAPI] valida firma + iss + aud + exp + roles --> Respuesta
```

Qué aporta esta implementación

- Cumple estándares OIDC + JWT RS256.
- Sin estado en la API (validación local por JWKS).
- Permite segregación de permisos por realm roles y client roles.
- Experiencia de prueba directa desde Swagger y Postman.
- Preparada para entornos con proxies/containers (public/internal issuer).

8. BONUS: Exportar configuración Keycloak

En esta sección, vamos a proponer una alternativa en ***Keycloak*** más elegante, relacionada con el utilizar una BBDD ya preconfigurada y montada en un volumen.

El objetivo es utilizar una configuración exportada de ***Keycloak***, y cuando se crea el contenedor con docker compose, importar dicha configuración (previamente exportada). Para ello, tenderemos que implementar una versión del archivo docker compose mejorada a la que llamaremos ../compose/docker-compose-keycloak-import.yml

Como paso preliminar, tengo que arrancar un contenedor de ***Keycloak***, y realizar las configuraciones que considere necesarias: realms, usuarios, roles, grupos, clientes, claves, flujos, ...etc. Una vez que tengamos todas las configuraciones realizadas, procedemos a exportar la configuración en un fichero ***.json***.

Para ello, utilizaremos este comando:

```
docker exec keycloak-service \
/opt/keycloak/bin/kc.sh export \
--realm fastapi-demo \
--dir /tmp/export \
--users realm_file
```

Este comando, crea un fichero ***.json*** con todas las configuraciones mencionadas, y preparadas para ser importadas. Este fichero se crea en la siguiente ubicación y con el siguiente nombre:

```
/tmp/export/fastapi-demo-realm.json
```

Y ahora procedo a copiar dicho fichero. Para ello, utilizaremos este comando:

```
docker cp keycloak-service:/tmp/export/fastapi-demo-realm.json ../keycloak-init/fastapi-demo-realm.json
```

La primera vez que creamos el contenedor, ejecutaremos un comando para importar el fichero ../keycloak-init/fastapi-demo-realm.json con toda la configuración.

El nuevo docker compose (../compose/dicker-compose-import.yml), quedaría de la siguiente forma:

```
services:
  postgres:
    image: postgres:15
    container_name: postgres-service-import
    restart: unless-stopped
    env_file:
      - .env
    environment:
      TZ: Europe/Madrid
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 60s
    # Connect to a common network
    networks: [appnet]

  keycloak:
    image: quay.io/keycloak/keycloak:26.5
    container_name: keycloak-service-import
    restart: unless-stopped
    env_file:
      - .env
    command:
      - start
      - --hostname-strict=false
      - --health-enabled=true
      - --http-enabled=true
      - --import-realm
      - --spi-import-export-dir=/opt/keycloak/data/impor
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://postgres:5432/${POSTGRES_DB}
      KC_DB_USERNAME: ${POSTGRES_USER}
      KC_DB_PASSWORD: ${POSTGRES_PASSWORD}
      # Optional: give more margin to the pool while Postgres starts
      QUARKUS_DATASOURCE_JDBC_ACQUISITION_TIMEOUT: 60S
      # Optional: bootstrap admin to log in directly
      #KC_BOOTSTRAP_ADMIN_USERNAME: ${KC_BOOTSTRAP_ADMIN_USERNAME}
```

```
      #KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_BOOTSTRAP_ADMIN_PASSWORD}
      KEYCLOAK_ADMIN: ${KEYCLOAK_ADMIN}
      KEYCLOAK_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD}
    volumes:
      - ../keycloak-init:/opt/keycloak/data/import
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
    # Optional: Keycloak healthcheck (when already connected to the DB)
    healthcheck:
      # TCP check: Open connection to 127.0.0.1:8080; if it fails, exit 1
      test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8080 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 30
      start_period: 60s
    # Connect to a common network
    networks: [appnet]

  app-fastapi:
    build:
      context: ../app
      dockerfile: Dockerfile
    container_name: app-fastapi-service
    restart: unless-stopped
    env_file:
      - .env
    depends_on:
      keycloak:
        condition: service_healthy
    ports:
      - "8001:8001"
    healthcheck:
      # TCP check: Open connection to 127.0.0.1:8001; if it fails, exit 1
      test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8001 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 30
      start_period: 60s
    # Connect to a common network
    networks: [appnet]

networks:
  appnet: {}
```

Para levantar los servicios, utilizamos el siguiente comando:

```
$ docker compose -f compose/docker-compose-keycloak-import.yml up
```

Y para apagar los servicios, procedemos con este otro:

```
$ docker compose -f compose/docker-compose-keycloak-import.yml down -v
```